

Data Pipeline

The data pipeline is a fundamental piece of Cass. It controls the flow of data into and out of an in-memory cache, your databases, the Riot API, and any other data sources/sinks you provide.

The data pipeline consists of a list of `DataSources` and `DataSinks`. A `DataSource` is any entity that *provides* data (for example, the Riot API and databases are both data sources). A `DataSink` is any entity that *stores* data (databases are also data sinks). Any entity that is a data sink will almost certainly be a data source as well. We refer to an entity that is both a data source and data sink as a *data store*.

The data sources and sinks are *ordered* in the data pipeline, and their order determines the order in which data is requested. Generally speaking, slower data stores / sinks should go towards the end of the pipeline.

For example, if your data pipeline consists of a cache, a database, and the Riot API (in that order), when you ask for a `Champion` Cassiopeia will first look in the cache, then in your database, then in the Riot API. If the data is found in the cache, it will be returned and the database and Riot API will not be queried. Similarly, if the data is found in the database, the Riot API will not be queried.

After data is found in a data source, the data propagates back down the data pipeline from whence it came. Any data sink encountered along the way will store that data. So, continuing the above example, if you asked for a `Champion` and it was provided by the Riot API, the champion data would be stored in your database, then stored in the cache. A data sink will only store data that it “accepts”. Cass’s built-in data sinks accept all of Cass’s data types.

Each data sink has expiration periods defined for each type of data it accepts. When data is put into a data sink, a clock starts ticking (metaphorically, programmatically this is handled differently). When that clock finishes, the data is expelled from the data sink. Static data should have an infinite expiration period (because it is stored per-version, and the static data for a given version never changes). Other types like `CurrentMatch` might have very short expiration periods. Each data sink defines its own default expiration periods, which are documented under the specific data sinks below.

A few notes: 1) Users can force all expired objects in data sinks to be removed using `settings.pipeline.expire()`. 2) Individual data sinks handle their own expirations, so if you write a database, you must decide how to handle expirations for data in your database.

Below is an example (which uses more datastores than Cass uses by default):

```
{
  "pipeline": {
    "Cache": {},

    "SimpleKVDiskStore": {
      "package": "cassiopeia_diskstore"
    },

    "DDragon": {},

    "RiotAPI": {
      "api_key": "RIOT_API_KEY"
    }
  }
}
```

 [latest](#) ▼

In brief, this means that the sequence for looking for data will be: 1) Look in the cache, 2) look in our disk-based database, 3) if it's static data, get it from data dragon, 4) pull the data from the Riot API.

Defining Components in your Settings

The components of the data pipeline are defined explicitly below, and you can choose which you want to use by setting the "pipelines" attribute in your settings. By default, Cass uses the in-memory cache, data dragon, and the Riot API.

Each component has it's own set of parameters, also described below.

[Settings](#) has an example data pipeline you can use in your settings if you want to modify the defaults.

Components

In-Memory Cache

The in-memory cache, simply called the cache, is a data store and provides fast read / write storage of data. It is used by including Cache in the data pipeline settings. If you are constantly creating the same data over and over, the cache is extremely useful. However, if you only using pulling a given piece of data once, it is likely unnecessary.

The cache should be the first element in your pipeline.

It takes one optional parameter (called `expirations`), which is a mapping of expiration times (in seconds or `datetime.timedelta` if set programmatically) for each data type stored in the cache. Valid type names and their defaults are below (a value of -1 means "do not expire" and 0 means "do not store in the data sink):

```
ChampionRotationData: datetime.timedelta(hours=6),
Realms: datetime.timedelta(hours=6),
Versions: datetime.timedelta(hours=6),
Champion: datetime.timedelta(days=20),
Rune: datetime.timedelta(days=20),
Item: datetime.timedelta(days=20),
SummonerSpell: datetime.timedelta(days=20),
Map: datetime.timedelta(days=20),
ProfileIcon: datetime.timedelta(days=20),
Locales: datetime.timedelta(days=20),
LanguageStrings: datetime.timedelta(days=20),
SummonerSpells: datetime.timedelta(days=20),
Items: datetime.timedelta(days=20),
Champions: datetime.timedelta(days=20),
Runes: datetime.timedelta(days=20),
Maps: datetime.timedelta(days=20),
ProfileIcons: datetime.timedelta(days=20),
ChampionMastery: datetime.timedelta(days=7),
ChampionMasteries: datetime.timedelta(days=7),
LeagueSummonerEntries: datetime.timedelta(hours=6),
League: datetime.timedelta(hours=6),
ChallengerLeague: datetime.timedelta(hours=6),
MasterLeague: datetime.timedelta(hours=6),
Match: datetime.timedelta(days=3),
```

 [latest](#) ▼

```

Timeline: datetime.timedelta(days=1),
Summoner: datetime.timedelta(days=1),
Account: datetime.timedelta(days=1),
ShardStatus: datetime.timedelta(hours=1),
CurrentMatch: datetime.timedelta(hours=0.5),
FeaturedMatches: datetime.timedelta(hours=0.5)

```

TODO: The cache currently does not automatically expire its data, so it's possible to run out of memory. To prevent this, users can trigger an expiration of all data or all data of one type by using the method `settings.pipeline.expire`. We will fix this so that the cache does automatically expire its data, but we haven't gotten to it yet. Using the `expire` method is a temporary workaround.

Data Dragon

Data Dragon is a data source and provides all of Cass's static data. This is largely due to the static data rate limits enforced by the Riot API. If you are testing your app and running it repeatedly without a database, you will need to continuously request the static data and will quickly hit the Riot API's rate limits. Data Dragon provides exactly the same data without some of the niceties that the Riot API provides.

Data Dragon should therefore come before the Riot API in your pipeline, but likely after your databases.

It takes no parameters (i.e. `{}`).

Riot API

Hopefully you already know what this is. It's where you're planning on getting your data, and it's a data source. It should come after your data bases, and will likely always be the last thing in your data pipeline.

This component can have complicated settings, so see [Settings](#) for its parameters.

Kernel

Cassiopeia can query a proxy server that mirrors Riot API endpoints. An example of such server is [Kernel](#).

To configure the address and ports of the proxy, use the following configuration within your pipeline:

```

{
  "pipeline": {
    ...,
    "kernel": {
      "server_url": "http://localhost",
      "port": 80
    }
    ...
  }
}

```

 [latest](#) ▼

Simple Disk Database

This is a simple filesystem database, and is therefore both a data source and data sink. It is not provided by Cass by default, and needs to be installed separately. See [plugins](#) for more information.

SQLAlchemy Database Support

This is a database system that supports all databases that [SQLAlchemy](#) supports. It is not provided by Cass by default, and needs to be installed separately. See [plugins](#) for more information.

Unloaded Ghost Store

As a user, it's very likely that you don't need to worry about what this store does. Cass automatically puts this store in your datapipeline.

The `UnloadedGhostStore` provides unloaded ghost objects to the rest of Cass when a new ghost object is created. This allows us to have a single location where all top-level objects are created, which alleviates some complicated issues that crop up when caching core objects and using ghost loading. In general, it should always be in your pipeline.

If you wish to override how Cass inserts it into your pipeline, you can include it in your pipeline and Cass won't insert it automatically. Normally, it should go immediately after the cache, and if you are not using a cache, it should be the first element in the data pipeline.